DASE 7506 · SEMESTER 1 · SESSION 3

Advanced Machine Learning Methods

The GPT architecture and pretraining — from 124 million parameters to a million-token context

Jiayu Chen Data and Systems Engineering, HKU jiayuc@hku.hk

16 SEPTEMBER

SESSION 3

Today's Three Blocks

01

Implementing a GPT Model

Chapter 4. Layer normalisation, GELU, the feed forward block, shortcut connections, and the 124-million-parameter model assembled.

50 MIN

02

Pretraining on Unlabelled Data

Chapter 5. Cross entropy and perplexity, the training loop, decoding strategies, and loading OpenAI's published weights.

50 MIN

03

Pretraining at the Frontier

Not in the book. RoPE, RMSNorm, SwiGLU, mixture of experts, and a close reading of the DeepSeek-V4 technical report.

50 MIN

Implementing a GPT Model from Scratch

Layer normalisation, GELU, shortcut connections, and the block that is repeated twelve times.

Where Chapter 4 Sits in the Build

Stage 1 is four steps. You have written the first two. Today we write the third and the fourth, and at the end of the session the model is a real GPT-2 with real weights.

STEP 1 · DONE

Data preparation

Byte-pair tokenization, sliding windows, token and positional embeddings.

CHAPTER 2

STEP 2 · DONE

Attention mechanism

Scaled dot product, causal masking, dropout, multi-head weight splits.

CHAPTER 3

STEP 3 · TODAY

LLM architecture

Normalisation, activations, the feed forward block, residual paths, the full model.

CHAPTER 4

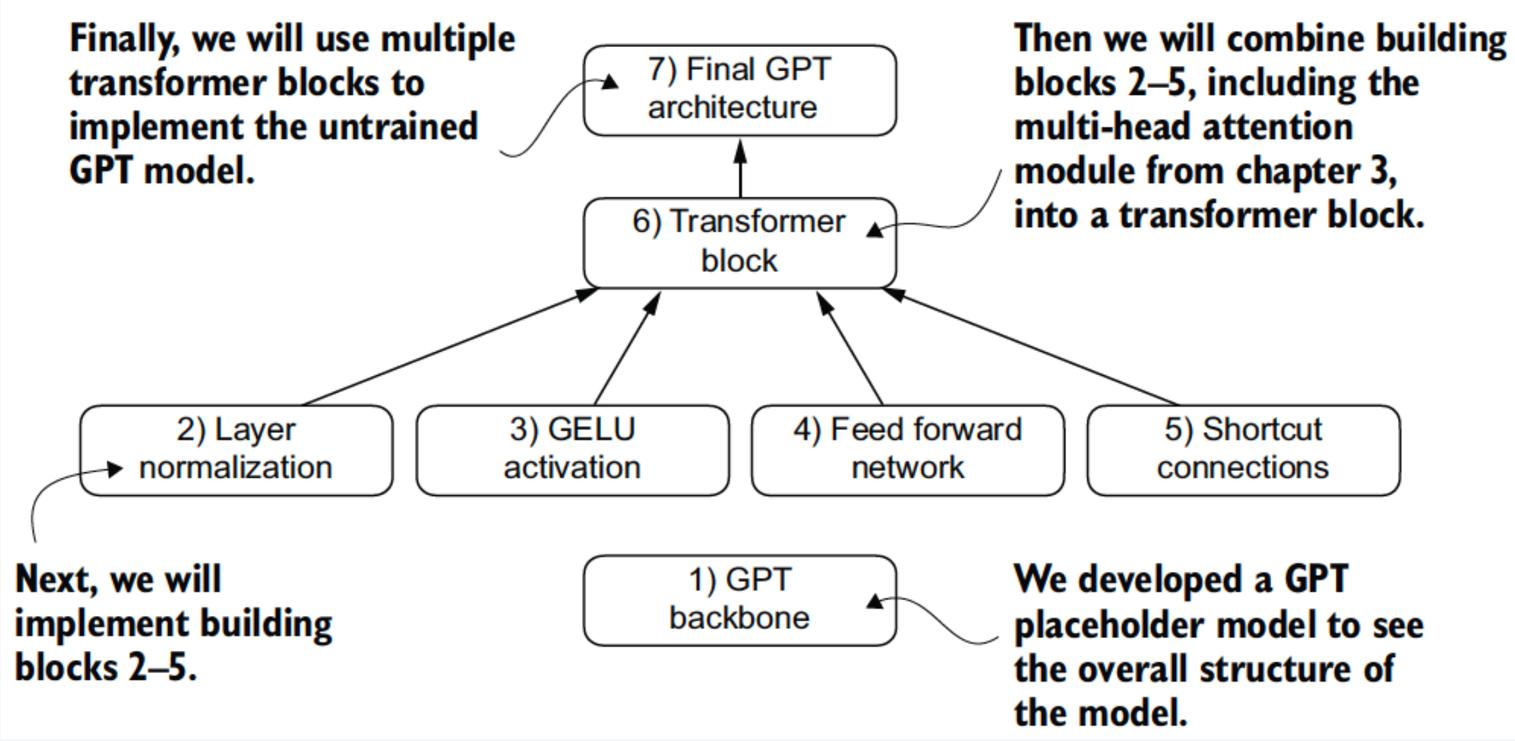
STEP 4 · TODAY

Pretraining

The training loop, the loss, decoding strategies, and loading published weights.

CHAPTER 5

Where Chapter 4 Sits in the Build



The Configuration of GPT-2 Small

```
GPT_CONFIG_124M = {
    "vocab_size":     50257,
    "context_length":  1024,
    "emb_dim":          768,
    "n_heads":           12,
    "n_layers":          12,
    "drop_rate":        0.1,
    "qkv_bias":       False
}
```

VOCAB_SIZE

Fixed by the BPE tokenizer from Chapter 2. It sets the size of both the input embedding and the output head.

CONTEXT_LENGTH

The number of learned positional embeddings. The model cannot see position 1025, because that row does not exist.

QKV_BIAS

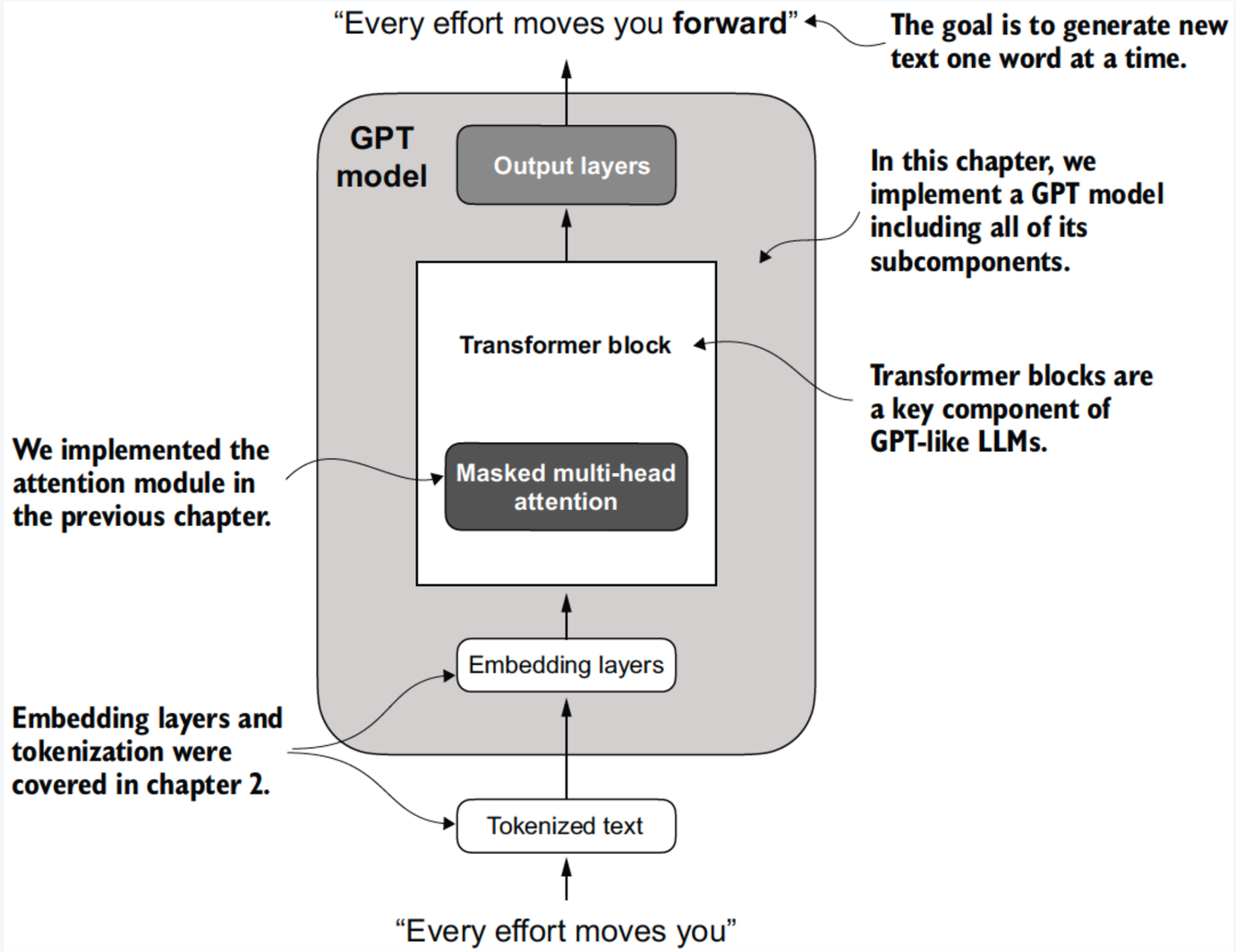
Disabled, following modern practice — but the original GPT-2 checkpoint has them, which matters in Block 2 when we load the weights.

One Architecture, Four Sizes

MODEL	EMB_DIM	N_LAYERS	N_HEADS	PARAMETERS
GPT-2 Small	768	12	12	124 M
GPT-2 Medium	1024	24	16	355 M
GPT-2 Large	1280	36	20	774 M
GPT-2 XL	1600	48	25	1558 M

CHAPTER 4.1

One Architecture, Four Sizes



CHAPTER 4.1

The Architecture, Top to Bottom

Tokenized text `[2, 4]`

Token embedding + positional embedding `[2, 4, 768]`

Dropout

× 12

Transformer block

LayerNorm → masked multi-head attention → dropout → residual, then LayerNorm → feed forward → dropout → residual

Final LayerNorm

Linear output head `[2, 4, 50257]`

GPT-3 is fundamentally the same in terms of model architecture, except that it is scaled up from 1.5 billion parameters in GPT-2 to 175 billion parameters in GPT-3, and it is trained on more data.

SHAPE IS INVARIANT

Every block takes `[b, t, 768]` and returns `[b, t, 768]`. That invariance is what makes the stack composable.

LOGITS, NOT PROBABILITIES

The model never applies a softmax. That belongs to the loss function in Block 2 and to the decoder at inference time.

A Backbone Made of Placeholders

```
class DummyGPTModel(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.tok_emb = nn.Embedding(cfg["vocab_size"], cfg["emb_dim"])
        self.pos_emb = nn.Embedding(cfg["context_length"], cfg["emb_dim"])
        self.drop_emb = nn.Dropout(cfg["drop_rate"])
        self.trf_blocks = nn.Sequential(
            *[DummyTransformerBlock(cfg)
              for _ in range(cfg["n_layers"])]
        )
        self.final_norm = DummyLayerNorm(cfg["emb_dim"])
        self.out_head = nn.Linear(
            cfg["emb_dim"], cfg["vocab_size"], bias=False
        )

    def forward(self, in_idx):
        batch_size, seq_len = in_idx.shape
        tok_embeds = self.tok_emb(in_idx)
        pos_embeds = self.pos_emb(
            torch.arange(seq_len, device=in_idx.device)
        )
        x = tok_embeds + pos_embeds
        x = self.drop_emb(x)
        x = self.trf_blocks(x)
        x = self.final_norm(x)
        logits = self.out_head(x)
        return logits
```

Uses a placeholder for TransformerBlock

Uses a placeholder for LayerNorm

CHAPTER 4.1

Shapes, End to End

INPUT

`[2, 4]`

Two texts, four token IDs each. Integers, not vectors.

HIDDEN STATE

`[2, 4, 768]`

Unchanged through all twelve blocks. The residual stream.

OUTPUT

`[2, 4, 50257]`

One logit per vocabulary entry, at every position.

The model predicts a next token at *every* position at once, not only at the last one. Four positions in, four independent predictions out — which is exactly what makes the training signal dense enough to learn from.

Why Deep Networks Need Normalisation

Activations drift in scale as they pass through layers. Once the scale of a layer's input depends on every weight below it, the gradient a layer receives depends on all of them too.

VANISHING

Repeatedly multiplying by factors below one shrinks the gradient towards zero. Early layers stop moving.

EXPLODING

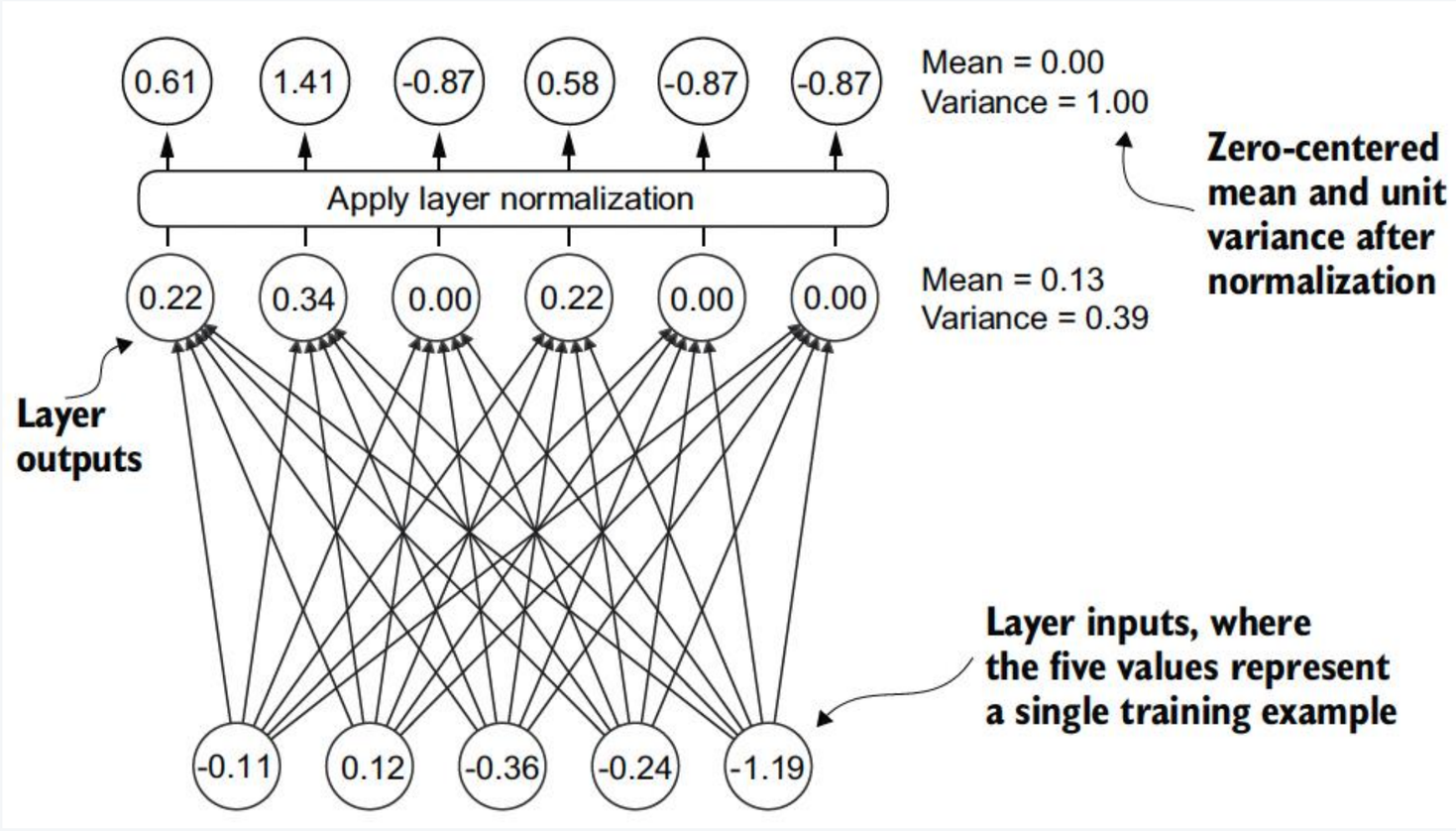
Factors above one compound the other way. The loss becomes `nan` and the run is dead.

THE FIX

Re-centre and re-scale the activations at every layer, so each layer sees an input distribution it can rely on.

Batch normalisation normalises across the batch; layer normalisation normalises across the features of one example.

Why Deep Networks Need Normalisation



MATHEMATICS

Layer Normalisation

$$y = \gamma \odot (x - \mu) / \sqrt{(\sigma^2 + \varepsilon)} + \beta, \mu = (1/d) \Sigma x_i, \sigma^2 = (1/d) \Sigma (x_i - \mu)^2$$

OVER WHAT

The last dimension only — the 768 features of one token. Batch and sequence positions are untouched, so nothing leaks between examples.

EPSILON

1e-5, added inside the square root. Without it, a token whose features are all equal divides by zero.

SCALE AND SHIFT

γ and β are trainable, one pair per feature. Normalisation is imposed; the layer is then free to learn its way back out.

γ and β can in principle invert the normalisation exactly, so the network can undo the operation.

Layer Normalisation in Eleven Lines

```
class LayerNorm(nn.Module):
    def __init__(self, emb_dim):
        super().__init__()
        self.eps = 1e-5
        self.scale = nn.Parameter(torch.ones(emb_dim))
        self.shift = nn.Parameter(torch.zeros(emb_dim))

    def forward(self, x):
        mean = x.mean(dim=-1, keepdim=True)
        var = x.var(dim=-1, keepdim=True, unbiased=False)
        norm_x = (x - mean) / torch.sqrt(var + self.eps)
        return self.scale * norm_x + self.shift
```

DIM = −1

The last axis, whatever the rank of the tensor. The same module works on `[b, d]` and `[b, t, d]`.

KEEPDIM = TRUE

Keeps the reduced axis at length one so broadcasting subtracts the right mean from the right token.

UNBIASED = FALSE

Divide by `n`, not `n−1`. At 768 features the difference is invisible, but it is what GPT-2's original implementation does — and weight compatibility is not negotiable.

CHAPTER 4.3

GELU Instead of ReLU

$$\mathrm{GELU}(x) = x \cdot \Phi(x) \approx 0.5 \cdot x \cdot [1 + \tanh(\sqrt{(2/\pi)} \cdot (x + 0.044715\, x^3))]$$

GATE, NOT SWITCH

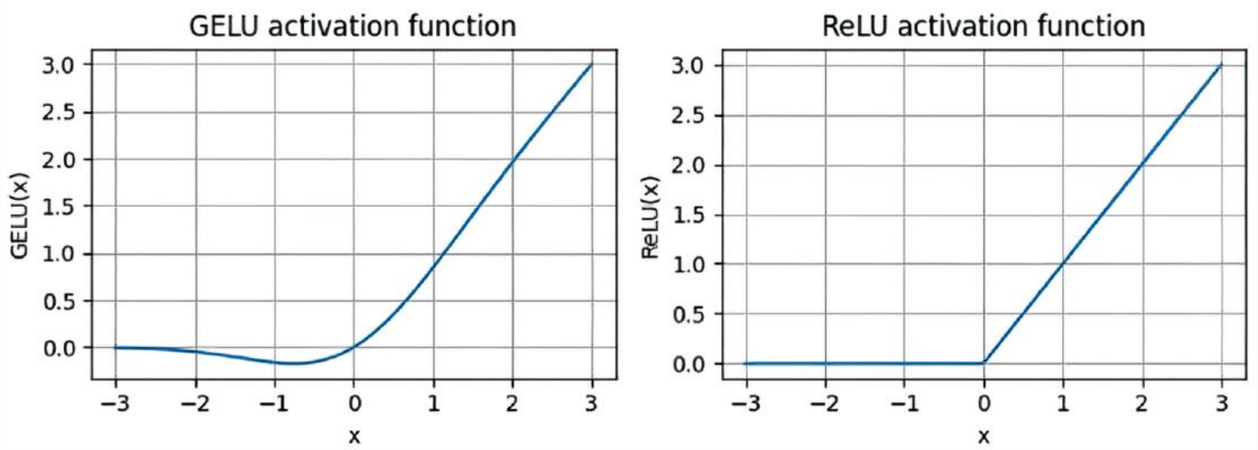
Φ is the standard normal CDF. The input is scaled by the probability that it exceeds a random threshold — a soft gate rather than a hard cut.

SMOOTH EVERYWHERE

Non-zero gradient for negative inputs, and no kink at the origin. Neurons that go negative can still recover.

NON-MONOTONIC

Slightly negative around x ≈ −0.75 before returning to zero, which gives the layer a way to express small negative responses.



The Feed Forward Module

```
self.layers = nn.Sequential(
    nn.Linear(cfg["emb_dim"], 4 * cfg["emb_dim"]),
    GELU(),
    nn.Linear(4 * cfg["emb_dim"], cfg["emb_dim"]),
)
```



POSITION-WISE

The same two matrices are applied independently to every token. No mixing across positions happens here — that is attention's job.

WHY EXPAND

A wider intermediate space gives the non-linearity room to separate features that are entangled at 768 dimensions, before projecting back.

PARAMETERS

2 × 768 × 3072 ≈ 4.72 M weights per block — twice what the attention module holds, and about two thirds of every transformer block.

Shortcut Connections and Gradient Flow

WITHOUT SHORTCUTS · MEAN |GRADIENT|

layers.0	0.00020
layers.1	0.00012
layers.2	0.00074
layers.3	0.00139
layers.4	0.00575

The first layer receives a gradient nearly thirty times smaller than the last. It barely trains.

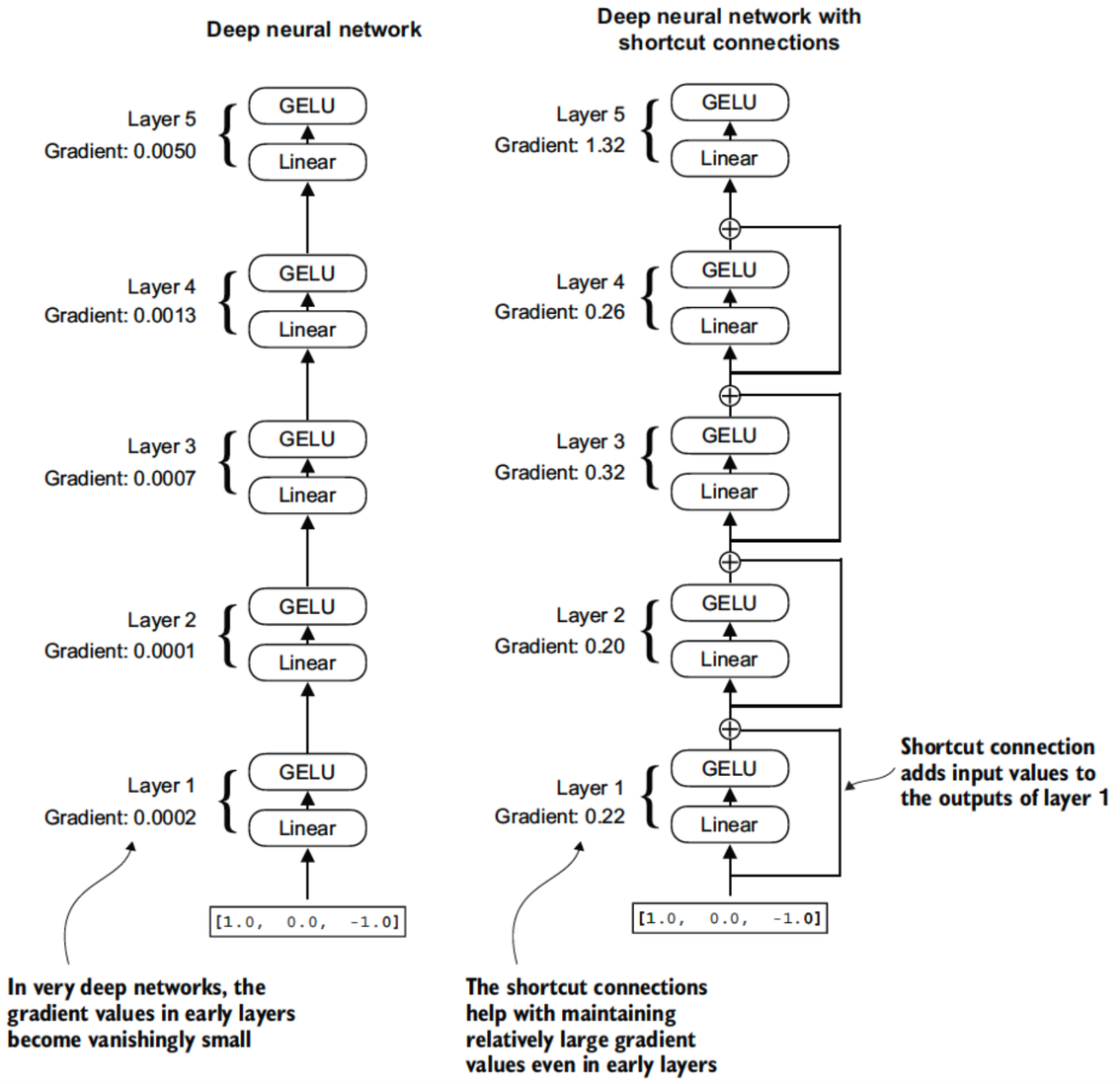
WITH SHORTCUTS · MEAN |GRADIENT|

layers.0	0.22169
layers.1	0.20694
layers.2	0.32889
layers.3	0.27077
layers.4	1.32600

Adding `x + F(x)` gives the gradient an identity path back to every layer.

CHAPTER 4.4

Shortcut Connections and Gradient Flow



The Transformer Block

```
def forward(self, x):
    shortcut = x
    x = self.norm1(x)
    x = self.att(x)
    x = self.drop_shortcut(x)
    x = x + shortcut

    shortcut = x
    x = self.norm2(x)
    x = self.ff(x)
    x = self.drop_shortcut(x)
    x = x + shortcut
    return x
```

PRE-NORM

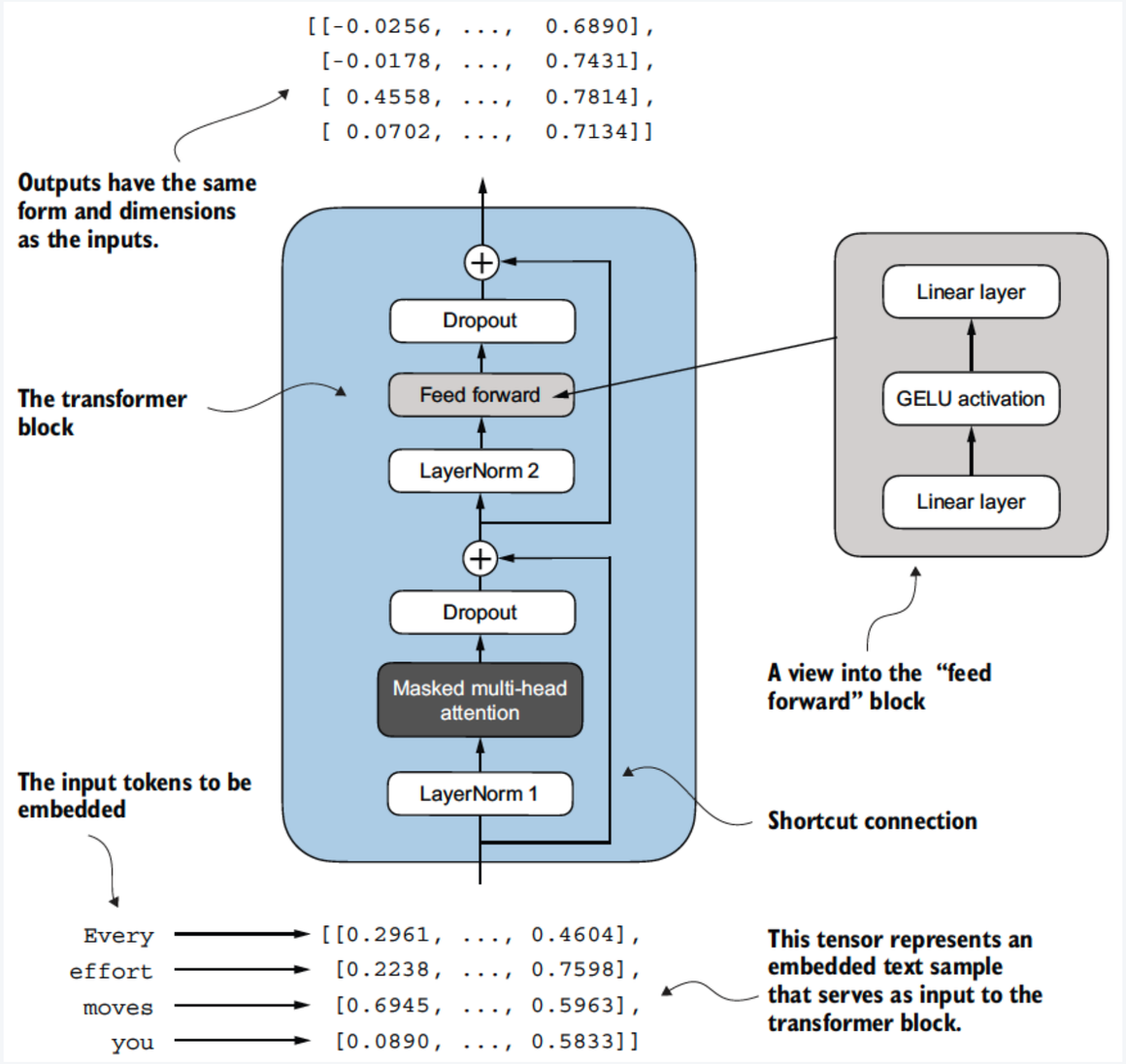
Normalisation happens inside the residual branch, never on the shortcut. The identity path stays untouched from the first block to the last.

TWO SUBLAYERS

Attention mixes information across positions; the feed forward network transforms each position on its own. Mixing and thinking, alternating twelve times.

Think: swap the order — residual add first, normalisation second. What happens to the scale of the residual stream after twelve blocks?

The Transformer Block



Where the Normalisation Goes

POST-NORM · VASWANI 2017

$$x \leftarrow \mathrm{LN}(\, x + F(x)\,)$$

The normalisation sits *on* the shortcut path, so the identity route back through the network is interrupted at every layer. Gradients at depth are unstable without a learning-rate warmup, and beyond about twenty layers the original recipe stops converging at all.

PRE-NORM · GPT-2 ONWARDS

$$x \leftarrow x + F(\, \mathrm{LN}(x)\,)$$

The normalisation moves inside the branch. The shortcut is now a clean identity from block one to block forty-eight, which is what lets deep stacks train — and it is why a final LayerNorm is needed before the output head.

THE COST OF PRE-NORM

Nothing bounds the residual stream, so its variance grows roughly linearly with depth and later blocks contribute proportionally less.

SANDWICH NORM

Normalise on the way in *and* on the way out of the branch, before the add. Keeps the clean shortcut and bounds what each block injects.

QK-NORM

A third location entirely: normalise queries and keys inside attention, which caps logit magnitude and prevents the attention entropy collapse seen at scale.

Think: pre-norm removed the need for warmup, yet every published pretraining run still uses one. What is the warmup doing that the normalisation placement does not?

The Model, Assembled

Listing 4.7 The GPT model architecture implementation

```
class GPTModel(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.tok_emb = nn.Embedding(cfg["vocab_size"], cfg["emb_dim"])
        self.pos_emb = nn.Embedding(cfg["context_length"], cfg["emb_dim"])
        self.drop_emb = nn.Dropout(cfg["drop_rate"])

        self.trf_blocks = nn.Sequential(
            *[TransformerBlock(cfg) for _ in range(cfg["n_layers"])])

        self.final_norm = LayerNorm(cfg["emb_dim"])
        self.out_head = nn.Linear(
            cfg["emb_dim"], cfg["vocab_size"], bias=False
        )

    def forward(self, in_idx):
        batch_size, seq_len = in_idx.shape
        tok_embeds = self.tok_emb(in_idx)

        pos_embeds = self.pos_emb(
            torch.arange(seq_len, device=in_idx.device)
        )
        x = tok_embeds + pos_embeds
        x = self.drop_emb(x)
        x = self.trf_blocks(x)
        x = self.final_norm(x)
        logits = self.out_head(x)
        return logits
```

The device setting will allow us to train the model on a CPU or GPU, depending on which device the input data sits on.

FINAL NORM

One more LayerNorm before the output head, so the logits do not inherit the drift of twelve accumulated residual additions.

163 Million, Not 124 Million

AS IMPLEMENTED

163,009,536

Token embedding and output head are two separate 768 × 50,257 matrices.

WITH WEIGHT TYING

124,412,160

One matrix, used twice — the original GPT-2 design, and where the name comes from.

The difference is a single 38.6-million-parameter matrix: 768 × 50,257. In a 124-million-parameter model, nearly a third of the weights are in the vocabulary projection.

Think: tying forces the vector that *reads* a token to be the vector that *predicts* it. Raschka reports that untied training performed better here. What does that tell you about what the two matrices are really learning?

What the Model Weighs

621.83 MB

163,009,536 parameters × 4 bytes, in 32-bit floating point. That is the smallest GPT-2, at rest, doing nothing.

+ GRADIENTS

One float per parameter, held for the whole backward pass. Double it.

+ OPTIMIZER STATE

AdamW keeps two moments per parameter. Double it again.

+ ACTIVATIONS

Scales with batch × sequence length, not with parameters. This is the term that explodes at long context.

Generating Text, One Token at a Time

```
def generate_text_simple(model, idx, max_new_tokens, context_size):
    for _ in range(max_new_tokens):
        idx_cond = idx[:, -context_size:]
        with torch.no_grad():
            logits = model(idx_cond)
        logits = logits[:, -1, :]
        probas = torch.softmax(logits, dim=-1)
        idx_next = torch.argmax(probas, dim=-1, keepdim=True)
        idx = torch.cat((idx, idx_next), dim=1)
    return idx
```

TRUNCATE THE CONTEXT

Keep only the last 1,024 tokens. Beyond that there is no positional embedding to look up.

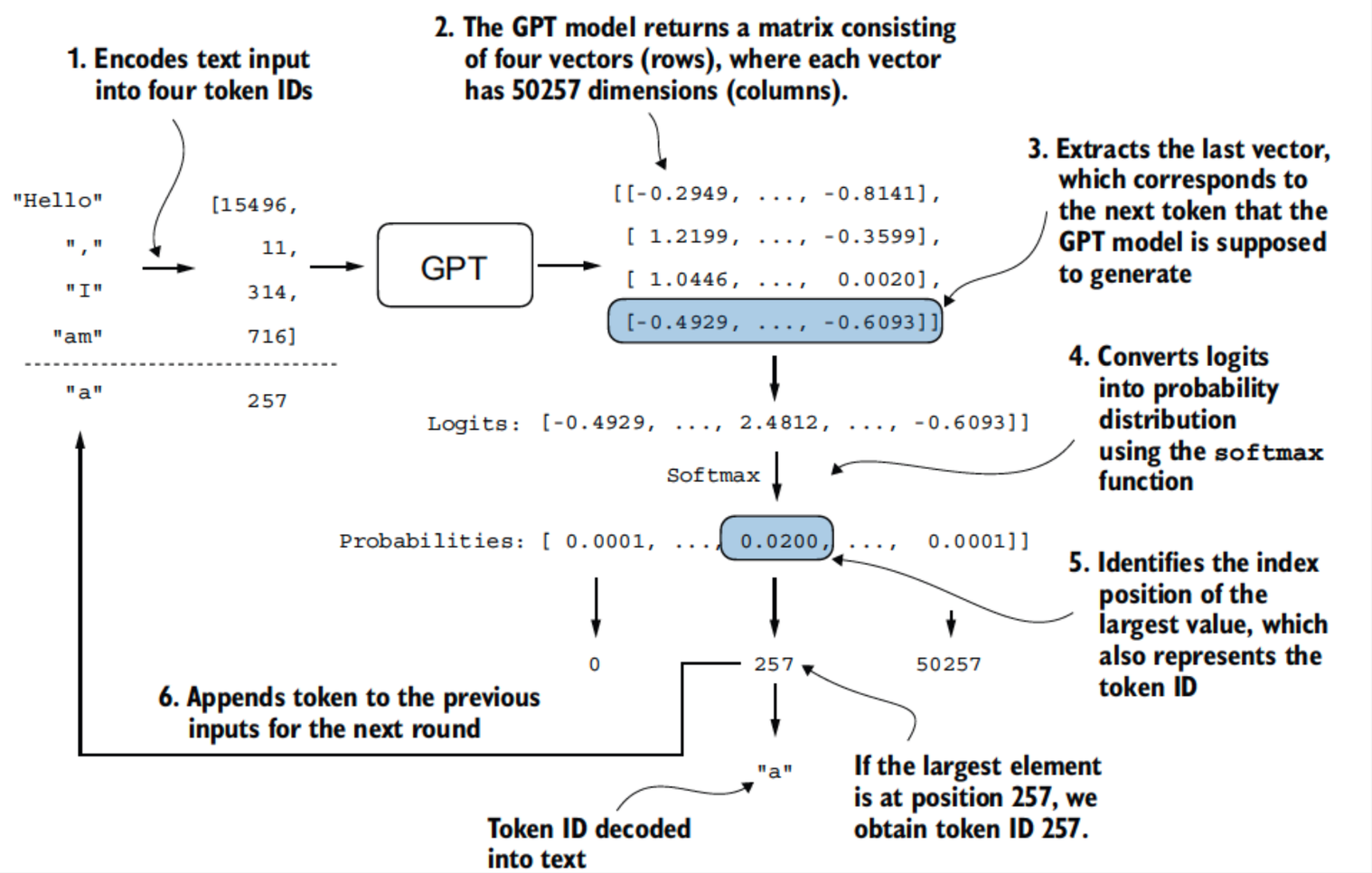
DISCARD ALL BUT ONE

The model returns a prediction at every position; generation throws away all of them except the last.

THE SOFTMAX IS FREE

Softmax is monotonic, so `argmax` over probabilities equals `argmax` over logits. It is here for clarity, and it is the hook where temperature will go in Block 2.

Generating Text, One Token at a Time



RECAP

Part I — The GPT Architecture

01 Layer normalisation acts across the 768 features of a single token, with a trainable scale and shift that let the layer opt back out.

02 GELU is a smooth gate rather than a hard switch; the feed forward module expands 768 to 3,072 and back, holding two thirds of each block's parameters.

03 Shortcut connections give the gradient an identity path to every layer — without them the first layer of a five-layer network sees a thousandth of the signal.

04 The transformer block is pre-norm, two sublayers, two residual additions, and shape-preserving — which is why depth is a single integer in the config.

05 The assembled model has 163 million parameters, weighs 622 MB in fp32, and generates fluent-looking nonsense.

Pretraining on Unlabelled Data

A loss, a loop, two decoding knobs, and someone else's weights.

The Five Things Chapter 5 Adds

01	A number for quality	Cross entropy loss and perplexity, computed from the logits you already produce.
02	Train and validation splits	Two loaders built from the sliding-window dataset of Chapter 2.
03	The training loop	Forward, loss, backward, step, evaluate — about thirty lines, and identical in every LLM ever trained.
04	Decoding strategies	Temperature scaling and top-k sampling, which trade determinism for diversity.
05	Loading published weights	OpenAI's GPT-2 checkpoints, mapped tensor by tensor into the class you wrote this morning.

Splitting the Corpus

```
train_ratio = 0.90
split_idx = int(train_ratio * len(text_data))

train_loader = create_dataloader_v1(
    text_data[:split_idx],
    batch_size=2,
    max_length=GPT_CONFIG_124M["context_length"],
    stride=GPT_CONFIG_124M["context_length"],
    drop_last=True,
    shuffle=True)
```

STRIDE = MAX_LENGTH

Non-overlapping windows. Overlap would recycle the same tokens as both input and target across batches, and inflate the apparent amount of data.

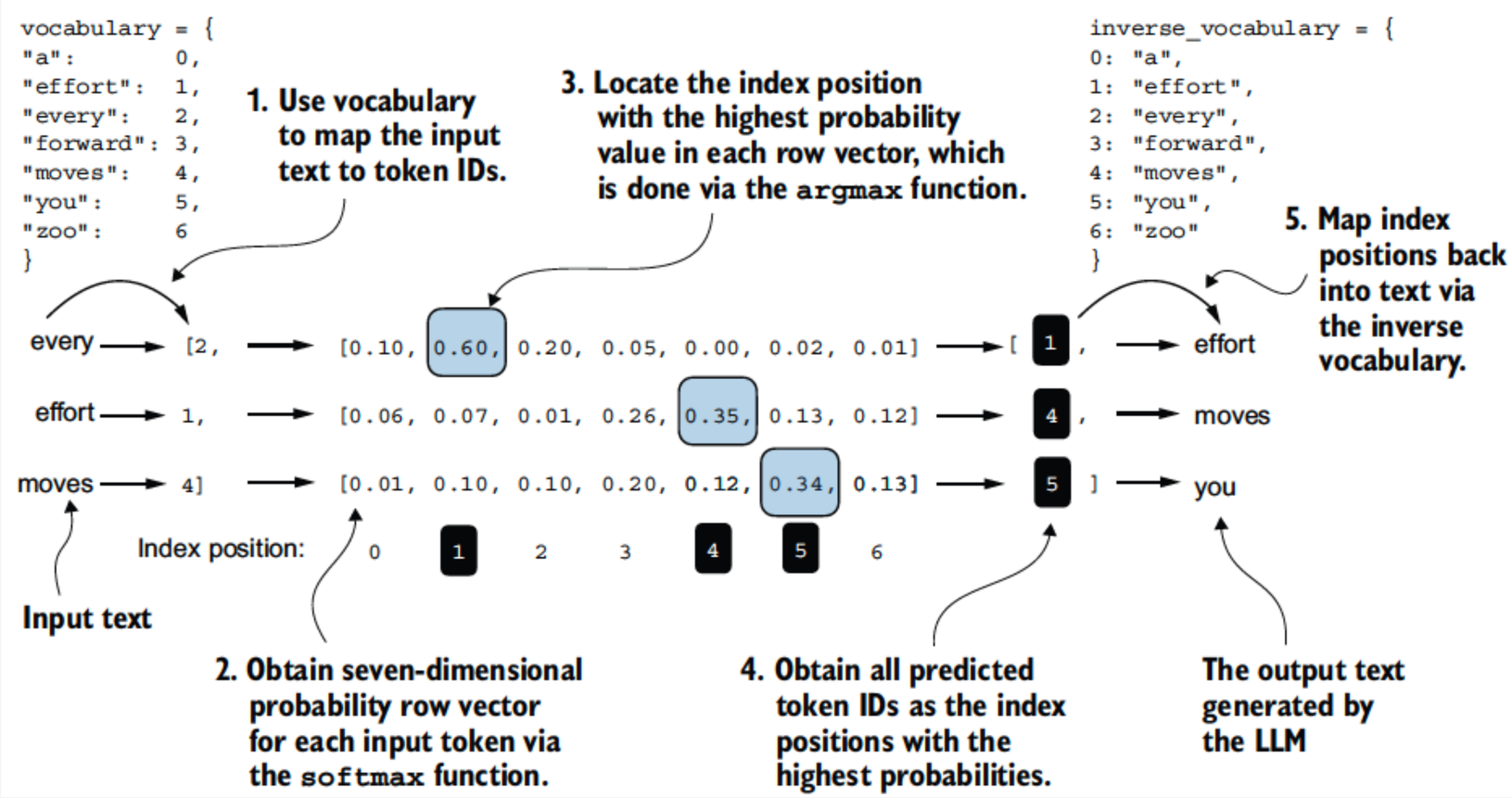
SPLIT BY CHARACTER

A contiguous cut, not a random shuffle of windows — otherwise the validation set contains sentences whose neighbours were trained on.

DROP_LAST

Discards the ragged final batch, so the loss is an average over batches of equal size and the numbers stay comparable.

Splitting the Corpus



From Logits to a Single Number

01 · LOGITS

The model's raw output, one score per vocabulary entry per position.

02 · PROBABILITIES

Softmax over the last axis turns the scores into a distribution.

03 · TARGET PROBABILITIES

Keep only the probability assigned to the token that actually came next.

04 · LOG

Take logarithms: products become sums, and tiny probabilities stop underflowing.

05 · AVERAGE

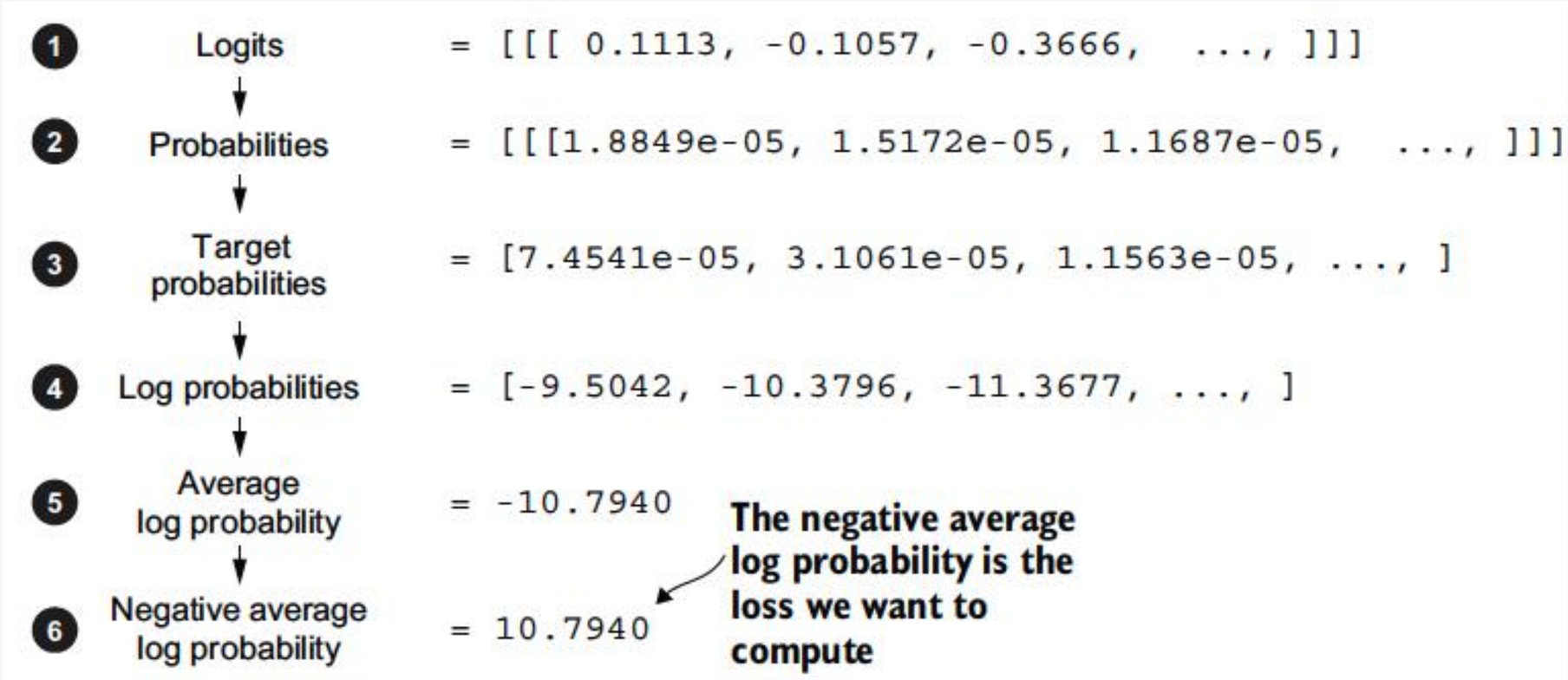
Mean over every position in the batch — one number for the whole batch.

06 · NEGATE

Flip the sign, so that gradient descent on the loss is ascent on the likelihood.

```
loss = torch.nn.functional.cross_entropy(logits.flatten(0, 1), targets.flatten())
```

From Logits to a Single Number



MATHEMATICS

Cross Entropy as Negative Log-Likelihood

$$L = -(1/N) \sum_t \log p_\theta(x_{t+1} \mid x_{\le t})$$

SELF-SUPERVISED

The label at position t is the token at position t+1. No annotation, no human in the loop — the corpus labels itself.

DENSE SIGNAL

Every position contributes a term. A batch of [2, 256] yields 512 supervised predictions, not two.

WHAT IT PENALISES

Confident errors, quadratically more than hesitant ones. A probability of 0.001 on the true token costs 6.9; a probability of 0.1 costs 2.3.

Perplexity

48,725 effective choices per token, for an untrained model with a 50,257-token vocabulary

$$\text{perplexity} = \exp(L) = \exp(10.7940) = 48{,}725.82$$

Perplexity is the loss made interpretable: the size of the vocabulary the model is effectively still undecided between at each step. A model that has learned nothing is perplexed by almost the entire vocabulary.

CHAPTER 5.2

The Training Set Is One Short Story

CHARACTERS	TOKENS	SPLIT
20,479	5,145	90 / 10

THE VERDICT

Edith Wharton, 1908, public domain. Nine training batches of `[2, 256]` and a single validation batch. The whole corpus fits in a text editor.

FOR SCALE

GPT-3 was pretrained on roughly 300 billion tokens; DeepSeek-V4 on more than 32 trillion.

Think: with 5,145 tokens and 163 million parameters, there are roughly 30,000 parameters per training token. Predict the shape of the validation loss curve before you see it.

Listing 5.3 The main function for pretraining LLMs

```
def train_model_simple(model, train_loader, val_loader,
                       optimizer, device, num_epochs,
                       eval_freq, eval_iter, start_context, tokenizer):
    train_losses, val_losses, track_tokens_seen = [], [], []
    tokens_seen, global_step = 0, -1

    for epoch in range(num_epochs):
        model.train()
        for input_batch, target_batch in train_loader:
            optimizer.zero_grad()
            loss = calc_loss_batch(
                input_batch, target_batch, model, device
            )
            loss.backward()
            optimizer.step()
            tokens_seen += input_batch.numel()
            global_step += 1

            if global_step % eval_freq == 0:
                train_loss, val_loss = evaluate_model(
                    model, train_loader, val_loader, device, eval_iter)
                train_losses.append(train_loss)
                val_losses.append(val_loss)
                track_tokens_seen.append(tokens_seen)
                print(f"Ep {epoch+1} (Step {global_step:06d}): "
                      f"Train loss {train_loss:.3f}, "
                      f"Val loss {val_loss:.3f}"
                )

        generate_and_print_sample(
            model, tokenizer, device, start_context
        )
    return train_losses, val_losses, track_tokens_seen
```

Initializes lists to track losses and tokens seen

Starts the main training loop

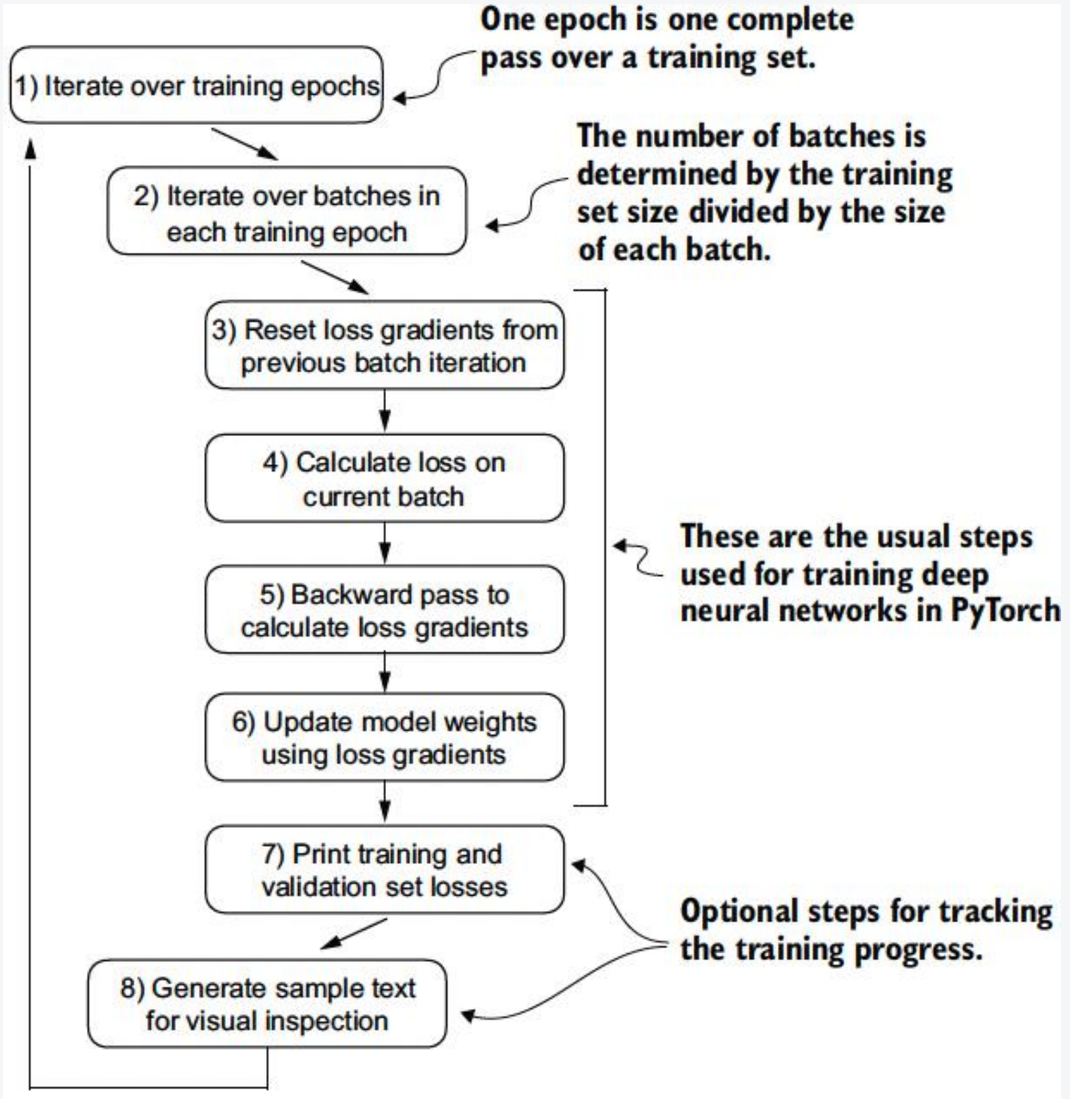
Resets loss gradients from the previous batch iteration

Calculates loss gradients

Updates model weights using loss gradients

Optional evaluation step

Prints a sample text after each epoch



The Training Loop

```
for epoch in range(num_epochs):
    model.train()
    for input_batch, target_batch in train_loader:
        optimizer.zero_grad()
        loss = calc_loss_batch(
            input_batch, target_batch, model, device)
        loss.backward()
        optimizer.step()
        tokens_seen += input_batch.numel()
```

ZERO_GRAD FIRST

PyTorch accumulates gradients by default. Forget this line and you are training on the sum of every batch you have ever seen.

MODEL.TRAIN()

Enables dropout. Evaluation switches it off, which is why train and validation losses are not measured under the same conditions.

TOKENS SEEN

The unit that matters. Frontier reports quote tokens, not epochs, because at scale nothing is ever seen twice.

CHAPTER 5.2

AdamW, and Why Not Adam

```
optimizer = torch.optim.AdamW(
    model.parameters(),
    lr=0.0004,
    weight_decay=0.1
)
```

$$\theta \leftarrow \theta - \eta \cdot \hat{m} / (\sqrt{\hat{v}} + \varepsilon) - \eta \cdot \lambda \cdot \theta$$

The decay term sits outside the adaptive scaling — that is the whole of the W.

TWO MOMENTS

A running mean and a running variance of the gradient, per parameter. Two extra floats per weight — the memory cost from the previous block.

DECOUPLED DECAY

In plain Adam, adding an L2 term to the gradient means the effective decay is divided by $\sqrt{\hat{v}}$ — parameters with noisy gradients get regularised less. AdamW applies λ directly to the weights instead.

CHAPTER 5.2

Ten Epochs, Five Minutes, One Laptop

STEP	TRAIN LOSS	VAL LOSS
000000	9.781	9.933
000005	8.111	8.339
000010	6.661	7.048
000040	3.220	6.371
000085	0.391	6.452

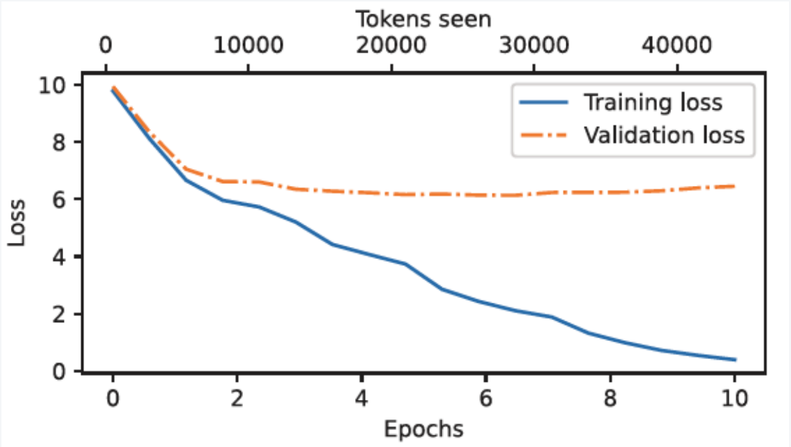
THE TWO CURVES SEPARATE

Training loss falls to 0.391; validation loss stalls near 6.4 and then rises. After roughly two epochs the model stops learning language and starts memorising Wharton.

THE SAMPLES AGREE

"Every effort moves you know," was one of the axioms he laid down across the Sevres and silver of an exquisitely appointed luncheon-table…

— reproduced verbatim from the training set.



THINK

Memorisation Is the Expected Result

PROBLEM 1

Frontier models train for a single epoch — every token is seen exactly once. Under what condition does one epoch on a large corpus dominate ten epochs on a small one, for a fixed compute budget?

PROBLEM 2

Validation loss here rises while the generated text becomes more fluent to read. Construct an argument that validation cross entropy is the wrong metric for this run — and then say what you would measure instead.

Temperature Scaling

$$p_i = \mathrm{softmax}(z_i / T)$$

T = 0.1

Near-deterministic. "forward" is chosen almost 100% of the time — the behaviour of argmax, reached by a smooth knob.

T = 1

The model's own distribution, untouched. "forward" about 60% of the time; "toward", "inches" and "closer" share the rest.

T = 5

Nearly uniform. "every effort moves you pizza" appears about 4% of the time. Diversity bought at the cost of sense.

Think: temperature rescales logits before the softmax, so it is a monotone reweighting — it can never make a token that ranked fourth rank first. What kind of generation failure can temperature therefore never fix?

Top-k Sampling

```
if top_k is not None:
    top_logits, _ = torch.topk(logits, top_k)
    min_val = top_logits[:, -1]
    logits = torch.where(
        logits < min_val,
        torch.tensor(float("-inf")).to(logits.device),
        logits)

if temperature > 0.0:
    logits = logits / temperature
    probs = torch.softmax(logits, dim=-1)
    idx_next = torch.multinomial(probs, num_samples=1)
else:
    idx_next = torch.argmax(logits, dim=-1, keepdim=True)
```

MASK WITH −INF

Not with zero. Softmax maps −∞ to exactly zero probability, and renormalises the survivors for free.

TRUNCATE, THEN SCALE

Top-k removes the tail; temperature reshapes what is left. Applying them in the other order changes which tokens survive.

FIXED k IS CRUDE

A confident step and an ambiguous step both keep k candidates. Nucleus sampling replaces the count with a probability mass — the natural next exercise.

Saving and Resuming a Run

```
torch.save({
    "model_state_dict": model.state_dict(),
    "optimizer_state_dict": optimizer.state_dict(),
}, "model_and_optimizer.pth")

checkpoint = torch.load("model_and_optimizer.pth")
model.load_state_dict(checkpoint["model_state_dict"])
optimizer.load_state_dict(checkpoint["optimizer_state_dict"])
```

STATE_DICT, NOT THE MODEL

A dictionary of tensors, decoupled from the class definition. Pickling the object instead ties your checkpoint to a particular file layout.

SAVE THE OPTIMIZER

AdamW's two moments per parameter are state. Resuming without them restarts the adaptive estimates cold, and the loss visibly jumps.

THE COST

The checkpoint is three times the size of the weights. At frontier scale this is why checkpointing cadence is a budget decision, not a habit.

Loading OpenAI's Published Weights

```
def assign(left, right):
    if left.shape != right.shape:
        raise ValueError(
            f"Shape mismatch. Left: {left.shape}, "
            f"Right: {right.shape}")
    return torch.nn.Parameter(torch.tensor(right))

gpt.pos_emb.weight = assign(gpt.pos_emb.weight, params["wpe"])
gpt.tok_emb.weight = assign(gpt.tok_emb.weight, params["wte"])
gpt.out_head.weight = assign(gpt.out_head.weight, params["wte"])
```

WTE, TWICE

The same tensor is assigned to the token embedding and to the output head — weight tying, made concrete. This is where the 163 million becomes 124 million.

QKV_BIAS = TRUE

The config has to be flipped back on before loading. GPT-2 has query, key and value biases; if your architecture does not, the checkpoint will not fit.

THE SHAPE CHECK IS THE POINT

Every architectural decision from Block 1 is now audited by an assertion. A silent mismatch would produce a model that runs and is wrong.

CHAPTER 5.5

The Same Code, 355 Million Trained Weights

THIS MORNING · RANDOM INITIALISATION

Every effort moves you rentingetic wasn? refres RexMeCHicular stren

NOW · GPT-2 MEDIUM, TOP-K 50, TEMPERATURE 1.5

Every effort moves you toward finding an ideal new way to practice something!

Nothing in your code changed. The architecture you wrote in Block 1 is bit-compatible with a checkpoint OpenAI trained in 2019 — which is the practical reason to build these models from scratch rather than call them.

RECAP

Part II — Pretraining

01 Cross entropy is the negative average log-probability of the observed next token; perplexity is its exponential, read as an effective vocabulary size.

02 An untrained model must start near ln(V) = 10.83. Check it first, every time.

03 The training loop is nine lines and does not change with scale; AdamW's decoupled decay and its two moments per parameter do set the memory budget.

04 Temperature reshapes the distribution, top-k truncates it; together they trade determinism for diversity, and neither can repair a badly ranked distribution.

05 Published weights load into your own class when — and only when — every shape and every convention matches.

Pretraining at the Frontier

Rotary embeddings, RMSNorm, SwiGLU, sparse experts — and a close reading of the DeepSeek-V4 technical report.

What Has Replaced Each Component

COMPONENT	GPT-2 · 2019	OPEN FRONTIER · 2026
Position	Learned absolute embedding	Rotary position embedding (RoPE)
Normalisation	LayerNorm, mean and variance	RMSNorm, scale only
Activation	GELU	SwiGLU, a gated linear unit
Feed forward	Dense, 4 × emb_dim	Sparse mixture of experts
Residual	x + F(x)	Manifold-constrained hyper-connections
Attention	Dense causal multi-head	Compressed sparse and heavily compressed, interleaved
Objective	Next token	Next token plus multi-token prediction
Optimiser	AdamW	Muon
Precision	fp32	FP4 quantisation-aware training

Three Problems with Learned Absolute Positions

```
self.pos_emb = nn.Embedding(cfg["context_length"], cfg["emb_dim"])
```

NO EXTRAPOLATION

The table has 1,024 rows. Position 1,025 has no vector, so the context length is welded into the parameter count at initialisation.

ABSOLUTE, NOT RELATIVE

Language cares about distance between tokens, not their index. The same phrase at position 12 and position 812 produces different embeddings, and the model must learn the invariance itself.

SPARSE GRADIENTS

Late positions appear in far fewer training sequences than early ones, so the tail of the table is systematically undertrained.

A million-token context is impossible with this line. The fix is not a bigger table — it is to stop adding position to the embedding at all, and to encode it inside the attention operation instead.

MATHEMATICS

Rotary Position Embedding

$$\tilde{q}_m = R_m\, q_m,\ \tilde{k}_n = R_n\, k_n,\ \theta_i = 10000^{-2i/d}$$

$$\langle\, R_m\, q,\ R_n\, k \,\rangle = \langle\, q,\ R_{n-m}\, k \,\rangle$$

ROTATION, NOT ADDITION

The head dimension is split into d/2 pairs; each pair is rotated in its own plane by $m\theta_i$. Norms are preserved, so nothing is added to the residual stream.

RELATIVE BY CONSTRUCTION

The attention score depends on m and n only through $n - m$. Relative position is a property of the algebra, not something the model has to learn.

A SPECTRUM OF FREQUENCIES

Low-index pairs rotate fast and resolve nearby tokens; high-index pairs rotate slowly and carry long-range order. One embedding, many scales.

Think: RoPE is applied to queries and keys but never to values. Why would rotating the values break the mechanism?

RoPE in Twenty Lines, With No Parameters

```
inv_freq = 1.0 / (base ** (torch.arange(0, d, 2) / d))
pos = torch.arange(context_length)
angles = pos[:, None] * inv_freq[None, :]      # [T, d/2]
angles = torch.cat([angles, angles], dim=-1)   # [T, d]
cos, sin = angles.cos(), angles.sin()

def apply_rope(x, cos, sin):
    x1, x2 = x[..., : d // 2], x[..., d // 2 :]
    rotated = torch.cat((-x2, x1), dim=-1)
    return x * cos + rotated * sin
```

ZERO PARAMETERS

Nothing here is learned. The 38.6-million-parameter position table from Block 1 becomes a cached pair of buffers.

APPLIED PER HEAD

Inside multi-head attention, after the q and k projections and before the dot product — not at the embedding layer.

THE BASE IS A KNOB

10,000 in the original paper; frontier long-context models raise it to 500,000 or beyond, which is the cheapest half of context extension.

RMSNorm and SwiGLU

$$y = \gamma \odot x \,/\, \sqrt{\left((1/d) \sum x_i^2 + \varepsilon \right)}$$

RMSNORM

Drop the mean subtraction and the shift; keep only the scale. One reduction instead of two, one parameter vector instead of two, and no measurable loss in quality at scale.

The implied claim is that re-centring never mattered — only re-scaling did.

$$\mathrm{FFN}(x) = \left(\mathrm{Swish}(x\, W_1) \odot x\, W_2 \right) W_3$$

SWIGLU

Three matrices instead of two. One branch produces a value, the other produces a multiplicative gate — the layer can now suppress a feature rather than merely fail to activate it.

To keep the parameter count fixed, the hidden width drops from 4d to about 8d/3.

Sparse Experts Instead of a Dense Feed Forward

DEEPSEEK-V4-PRO · TOTAL

1.6 T

parameters held in memory across the expert pool

ACTIVATED PER TOKEN

49 B

parameters that actually multiply — about 3% of the model

THE ROUTER

A small linear layer scores each expert per token and sends the token to the top few. Capacity grows with the pool; compute grows only with the number selected.

FINE-GRAINED AND SHARED

DeepSeekMoE splits the pool into many narrow experts and keeps a few always-on shared ones, so common knowledge is not duplicated in every expert.

LOAD BALANCE

Left alone, routers collapse onto a few favourites. V4 tunes the affinity score and removes the cap on routing targets; the earliest blocks use hash routing instead.

Towards Highly Efficient Million-Token Context

V4-PRO

1.6 T · 49 B active

V4-FLASH

284 B · 13 B active

CONTEXT

1 M

PRETRAINING

32 T+

FLOPS VS V3.2

27%

KV CACHE VS V3.2

10%

The claim of the paper is not raw capability. It is the efficiency-per-context-token curve: at a million tokens, V4-Pro needs roughly a quarter of the inference FLOPs and a tenth of the KV cache of its own predecessor.

The V4 Block, Against Yours

GPT-2 BLOCK · THIS MORNING	DEEPSEEK-V4 BLOCK
LayerNorm	RMSNorm
Masked multi-head attention	Hybrid attention — CSA or HCA, interleaved by layer
Residual add	Manifold-constrained hyper-connections
LayerNorm	RMSNorm
Dense feed forward, GELU	DeepSeekMoE feed forward
Residual add	Multi-token prediction heads at the output

Four Ways to Avoid the Quadratic Matrix

FEWER KV	Grouped-query attention	Heads share key and value projections. Cache shrinks by the group factor; nothing else changes.
SMALLER KV	Multi-head latent attention	Keys and values are stored in a low-rank latent space and reconstructed on the fly. DeepSeek-V2 through V3.2.
READ LESS	Sparse and sliding-window attention	Attend to a selected subset of positions rather than all of them. Selection itself has to be cheap — hence an indexer.
SHORTEN T	Compression, then sparsity	V4's move: shrink the sequence the attention operates over, then apply sparsity to what remains.

Manifold-Constrained Hyper-Connections

BLOCK 1

$$x_{l+1} = x_l + F(x_l)$$

A single residual stream, one fixed identity path.

HYPER-CONNECTIONS

$$X_{l+1} = B_l X_l + F(X_l)$$

Several parallel streams, mixed by a learned matrix at every layer.

THE INSTABILITY

A free B can have spectral norm above one. Compose hundreds of layers and the residual stream diverges — which is why plain hyper-connections could not be stacked deeply.

THE CONSTRAINT

Project B onto the Birkhoff polytope — the doubly stochastic matrices — by Sinkhorn–Knopp iteration. Every row and column sums to one.

THE CONSEQUENCE

Spectral norm is bounded by one, so the map is non-expansive and signal propagation stays stable across a 1.6-trillion-parameter graph.

Muon, and Training in Four Bits

MUON, REPLACING ADAMW

AdamW treats every weight as an independent scalar. Muon treats a weight matrix as a matrix: it takes the momentum update and orthogonalises it by Newton–Schulz iteration before applying it.

The effect is that no single direction in the update dominates, which the report credits with faster convergence and greater stability. V4 is the first deployment of Muon on a trillion-plus-parameter mixture of experts, paired with a custom hybrid ZeRO strategy.

FP4, DURING TRAINING

Not post-hoc inference quantisation. Four-bit quantisation is applied inside the training loop, to the mixture-of-experts weights and to the indexer's query-key path, so the model learns weights that survive the precision it will be served at.

PER PARAMETER

fp32 → 4 bytes

fp4 → half a byte

Compare the 621.83 MB from Block 1: the same model in fp4 would be 20 MB of weights.

MATHEMATICS

Inside the Muon Update

$$M_t = \mu\, M_{t-1} + G_t \rightarrow O_t = \mathrm{NewtonSchulz}(\, M_t\,) \rightarrow W_t = W_{t-1} - \eta\, O_t$$

$$\mathrm{NewtonSchulz}(M) \approx U\, V_T, \text{ where } M = U\, \Sigma\, V_T$$

THROW AWAY Σ

Orthogonalising the momentum matrix keeps its singular *directions* and sets every singular value to one. The update's shape survives; its conditioning does not.

NO SVD

Five iterations of a quintic polynomial in M — matrix multiplies only, in bf16. An exact SVD every step would cost more than the forward pass.

ONE MOMENT, NOT TWO

Muon carries a single momentum buffer per matrix. AdamW carries two. Embeddings, the output head and every bias stay on AdamW — Muon applies to 2-D weights only.

Think: AdamW's per-parameter scaling makes the update's largest singular direction dominate as training proceeds. Muon removes that dominance by construction — so why is it still the wrong optimiser for the embedding table?

Many Specialists, One Generalist

01 · SPECIALISE

Train an independent expert model per domain — mathematics, code, agentic tool use, instruction following — each with supervised fine-tuning followed by GRPO.

02 · CONSOLIDATE

Merge the specialists into a single model by on-policy distillation with a reverse-KL objective: the student samples, the teachers score, the student is pulled towards them.

03 · THE SUBSTITUTION

Reinforcement learning is confined to producing the specialists. The final consolidation step replaces RL entirely — a notable departure from the V3 recipe.

Reverse KL is mode-seeking: the student is punished for putting mass where the teacher has none, but not for ignoring parts of the teacher's distribution. The result is a narrower, more confident model — which is what you want from a generalist assembled out of specialists.

GRPO, reward models and distillation are Part II of this course, lectures 5 to 8.

READING

Before Session 4

REQUIRED	Raschka, chapters 4 and 5	Including the exercises. Run the notebooks.
REQUIRED	DeepSeek-V4 technical report	arXiv 2606.19348. Sections on hybrid attention and mHC.
RECOMMENDED	Su et al., RoFormer	arXiv 2104.09864. The original rotary embedding paper.
RECOMMENDED	Shazeer, GLU Variants Improve Transformer	arXiv 2002.05202. Two pages, and the reason for SwiGLU.
RECOMMENDED	Peng et al., YaRN	arXiv 2309.00071. How context windows are actually extended.

Stage 1 is finished. You have a GPT, and you have trained one.

Session 4 turns a base model into an assistant: classification fine-tuning, instruction fine-tuning, parameter-efficient adaptation with LoRA, and preference tuning.